

No. Operator Questions

What is the operator?

What is membership operators?

What is Identity operators?

Difference between '==' and 'is' Operator?

What is ternary operator in Python?

Ques. What is the operator?

- Ques. What is the operator?
- **Arithmetic Operators**
- **Comparison(Relational) Operator**
- **Assignment Operators**
 - **Bitwise Operators**
- **What is membership operators?**
 - in operator
- **What is Identity operators?**
 - is operator
 - is not operator
- **What is ternary operator in Python?**

Arithmetic Operators

Operators	Description	Result
Addition(+)	Adds the values on either side of the operator.	$3+4=7$
Subtraction(-)	Subtracts the value on the right from the one on the left.	$3+4=-1$
Multiplication(*)	Multiplies the values on either side of the operator.	$3*4=12$
Division(/)	Divides the value on the left by the one on the right. Notice that division results in a floating-point value.	$3/4=0.75$
Exponentiation(**)	Raises the first number to the power of the second.	$3**4=81$
Floor Division(//)	Divides and returns the integer value of the quotient. It dumps the digits after the decimal.	$10//3=3$
Modulus(%)	Divides and returns the value of the remainder.	$3\%4=3$

Comparison(Relational) Operator

Operators	Description	Result
Less than(<)	This operator checks if the value on the left of the operator is lesser than the one on the right.	3<4=True
Greater than(>)	It checks if the value on the left of the operator is greater than the one on the right.	3>4=False
Less than or equal to(<=)	It checks if the value on the left of the operator is lesser than or equal to the one on the right.	7<=7 = True
Greater than or equal to(>=)	It checks if the value on the left of the operator is greater than or equal to the one on the right.	0>=0 = True
Equal to(==)	This operator checks if the value on the left of the operator is equal to the one on the right.(1 is equal to the Boolean value True, but 2 isn't. Also, 0 is equal to False.)	3==3.0 = True
		1==True = True
		7==True = False
		0==False = True
		0.5==True = False

Assignment Operators

- Assignment operators

Operator	Example	Same As	Try it
=	x = 5	x = 5	x = 5 print(x) 5
+=	x += 3	x = x + 3	x = 5 x += 3 print(x) 8
-=	x -= 3	x = x - 3	x = 5 x -= 3 print(x) 2
*=	x *= 3	x = x * 3	x = 5 x *= 3 print(x) 15
/=	x /= 3	x = x/3	x = 5 x /= 3 print(x) 1.66

```
%= x %= 3 x = x % 3
//= x //= 3 x = x // 3
**= x **= 3 x = x ** 3
&= x &= 3 x = x & 3
|= x |= 3 x = x | 3
^= x ^= 3 x = x ^ 3
>>= x >>= 3 x = x >> 3
<<= x <<= 3 x = x << 3
```

Operators	Description	Result
Assign(=)	Assigns a value to the expression on the left. Notice that = = is used for comparing, but = is used for assigning.	>>> a=7 >>> print(a) //output:- 7
Add and Assign(+ =)	Adds the values on either side and assigns it to the expression on the left. a+=10 is the same as a=a+10.	>>> a+=2

Operators	Description	Result
<pre>>>> print(a) //output:- 9</pre>		
Divide and Assign(/=)	Divides the value on the left by the one on the right. Then it assigns it to the expression on the left.	<pre>>>> a/=7 >>> print(a) //output:- 1.0</pre>
Multiply and Assign(*=)	Multiplies the values on either sides. Then it assigns it to the expression on the left.	<pre>>>> a*=8 >>> print(a) // 8.0</pre>
Modulus and Assign(%=)	Performs modulus on the values on either side. Then it assigns it to the expression on the left.	<pre>>>> a%=3 >>> print(a) //output:- 2.0</pre>
Exponent and Assign(**=)	Performs exponentiation on the values on either side. Then assigns it to the expression on the left.	<pre>>>> a**=5 >>> print(a) //output:- 32.0</pre>
Floor-Divide and Assign(//=)	Performs floor-division on the values on either side. Then assigns it to the expression on the left.	<pre>>>> a//=3 >>> print(a) //output:- 10.0</pre>

Bitwise Operators

Operators	Description	Result
Binary AND(&)	It performs bit by bit AND operation on the two values. Here, binary for 2 is 10, and that for 3 is 11. &-ing them results in 10, which is binary for 2.	>>> 2&3 //output:- 2
Binary OR()		--
Binary XOR(^)	--	--
Binary One's Complement(~)	--	--
Binary Left-Shift(<<)	--	--
Binary Right-Shift(>>)	--	--

What is membership operators?

- A membership operator in Python checks if a value or variable is present in a sequence(string, list, tuples, sets, dictionary) or not.
- The output of a membership operator is a Boolean value, either True or False.
- There are two membership operators in Python.

1. In operator
2. not in' operator

in operator

- The 'in' operator is used to check if a value exists in a sequence or not.

```
#in
x = "Hello, World!"
print("ello" in x) #Returns true as it exists
print("hello" in x) #Returns false as 'h' is in lowercase
print("World" in x)#Returns true as it exists
```

```
x = ["apple", "banana"]
print("banana" in x) # Output:- True
```

```
list1=[0,2,4,6,8]
list2=[1,3,5,7,9]

check=0
for item in list1:
    if item in list2:
        #Overlapping true so check is assigned 1
        check=1

if check==1:
    print("overlapping")
else:
    print("not overlapping")
```

Output:- not overlapping

2. **'not in operator'**- Returns True if the target value is not present in a given collection of values. Otherwise, it returns False.

```
#not in
x = "Hello, World!"
print("ello" not in x) #Returns false as it exists
print("hello" not in x) #Returns true as it does not exist
print("World" not in x) #Returns false as it exists
```

```
x = ["apple", "banana"]
print("pineapple" not in x) # Output:- True
```

```
a = 70
b = 20
list = [10, 30, 50, 70, 90 ];

if ( a not in list ):
    print("a is NOT in given list")
else:
    print("a is in given list")

if ( b not in list ):
    print("b is NOT present in given list")
else:
    print("b is in given list")

Output:-
a is in given list
b is NOT present in given list
```

What is Identity operators?

- The Python Identity Operators are used to compare the objects if both the objects are actually of the **same data type** and share the **same memory location**. There are different identity operators such as:
There are two Identity operators in Python.

1. is operator
2. is not operator

is operator

- is operator returns True if both variables refer to the same object, otherwise, it returns False.

```
x = 'Educative'
if (type(x) is str):
    print("true")
else:
    print("false")
Output:- True
```

is not operator

- is not operator returns True if both variables do not refer to the same object, otherwise, it returns False.

```
x = 6.3
if (type(x) is not float):
    print("true")
else:
    print("false")
```

Output:- False

```
# is not Operator:
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x

print(x is not z)    # Output:- False
print(x is not y)    # Output:- True
print(x != y)        # Output:- False
```

Difference between '==' and 'is' Operator?

- The **equality operator(==)** is used to compare the value of two variables,
- whereas the identities operator **is** used to compare the memory location of two variables.

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)    # Output:- True, because the contents are the same

# -----identities operator-----
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a is b)    # False, different objects in memory
print(a is c)    # True, both refer to the same object
```

What is ternary operator in Python?

- The ternary operator in Python is a **one-line conditional expression** used to assign a value based on a condition.

```
a = 10
b = 20

max_value = a if a > b else b
print(max_value) # Output:- 20
```

```
def check(x):
    return "Positive" if x > 0 else "Non-positive"

print(check(-5))
```

- Nested Ternary Operator

```
num = 0

result = "Positive" if num > 0 else "Negative" if num < 0 else "Zero"
print(result) # Output:= Zero
```